

UNIVERSIDAD DE GUAYAQUIL
INGENIERÍA DE SOFTWARE

MANUAL TÉCNICO

SISMOAPP

PROYECTO DE VINCULACIÓN
EVALUACIÓN DE LA VULNERABILIDAD SÍSMICA
DE LAS EDIFICACIONES UBICADAS EN LAS PARROQUIAS
OLMEDO Y BOLIVAR DE LA CIUDAD DE GUAYAQUIL

GUAYAQUIL – ECUADOR

AGOSTO 2025

TABLA DE CONTENIDO

1	Introducción	4
2	Guía de Conexión: Servicio de Autenticación al Servidor	5
2.1	Backend – Servidor API.....	5
2.2	Frontend – Flutter.....	5
3	Paso a paso de la conexión.....	6
3.1	Backend.....	6
3.2	Frontend	6
3.3	Problemas de conexión.....	7
3.3.1	Análisis del error de conexión	8
4	Pasos para la configuración de la conexión	8
4.1	PASO 1: Verificación de Configuración del Servidor.....	8
4.1.1	Verificar configuración de IP en DatabaseConfig.....	8
4.1.2	Obtener la IP correcta de tu computadora.....	9
4.2	PASO 2: Verificación del Servidor Backend.....	10
4.2.1	Confirmar que el servidor está ejecutándose	10
4.2.2	Identificar tu arquitectura actual	11
4.2.3	Verificar el servidor personalizado	12
4.2.4	Probar endpoint de health	14
4.3	PASO 3: Configuración de Red y Firewall	15
4.3.1	Verificar conectividad de red	15
4.3.2	Verificar que el puerto 3000 esté abierto	15
4.3.3	Configurar Firewall.....	16

4.4	PASO 4: Prueba de Conexión desde la Aplicación	16
4.4.1	Usar el widget ConnectionTestAndroid	16
4.4.2	Interpretar los resultados de la prueba	17
4.5	PASO 5: Solución de Problemas Comunes	18
4.5.1	Error "No route to host"	18
4.5.2	Error "Connection timeout"	19
4.5.3	Error "Connection refused"	20
4.6	PASO 6: Validación Final de la Conexión	20
4.6.1	Prueba completa del flujo de autenticación	20
4.7	Casos de prueba.....	21
4.7.1	Prueba 1: Conexión exitosa	21
4.7.2	Prueba 2: Servidor apagado	21
4.7.3	Prueba 3: IP incorrecta	21
4.8	PASO 7: Monitoreo y Logs	22
4.8.1	Habilitar logs detallados	22
4.8.2	Verificar logs del servidor.....	22
4.8.3	Conexión exitosa.....	24

1 INTRODUCCIÓN

El presente manual técnico tiene como finalidad explicar de manera clara y estructurada la conexión entre el **frontend** desarrollado en **Flutter** y el **backend** implementado como una **API REST** en **Node.js + Express**. La correcta comunicación entre ambas capas es esencial para garantizar que los datos viajen de forma segura, consistente y confiable, permitiendo que la aplicación móvil consuma los servicios y muestre la información al usuario final.

En este documento se describen los componentes principales del sistema:

- **Backend (API REST):** encargado de exponer servicios a través de endpoints que responden en formato JSON.
- **Frontend (Flutter):** la aplicación móvil que consume dichos servicios mediante peticiones HTTP.

Asimismo, se detallan los pasos necesarios para configurar la conexión, el flujo de datos entre el cliente y el servidor, y las consideraciones técnicas que deben tenerse en cuenta según el entorno de ejecución (emulador Android, dispositivo físico o navegador).

Finalmente, se incluyen los posibles errores de conexión más comunes, como el caso de “ClientException: Failed to fetch” o “No route to host”, junto con sus causas más probables y las soluciones recomendadas, tales como ajustes en la dirección IP, el puerto, la configuración de red y la correcta distinción entre **HTTP** y **HTTPS**.

Este manual busca servir como guía de referencia para desarrolladores y personal técnico, facilitando la integración del frontend con el backend y asegurando una comunicación estable durante el desarrollo, pruebas y despliegue de la aplicación.

2 GUÍA DE CONEXIÓN: SERVICIO DE AUTENTICACIÓN AL SERVIDOR

2.1 Backend – Servidor API

El backend está en **Node.js + Express**.

Arranca con:

- `npm install`
- `npm start`

Se levanta en una dirección como:

- `http://localhost:3000`

Define **endpoints**:

- `GET /api/buildings` → lista edificios.
- `POST /api/buildings` → crea un edificio.
- `PUT /api/buildings/:id` → actualiza un edificio.
- `DELETE /api/buildings/:id` → elimina un edificio.

Cada endpoint devuelve datos en formato **JSON** que el frontend interpreta.

2.2 Frontend – Flutter

En el proyecto Flutter existe algunos service, para propósitos de la explicación usaremos

`building_service.dart` que hace las llamadas HTTP al backend usando `http` o `dio`.

```

class BuildingService {
  // CREATE BUILDING con archivos multimedia
  static Future<BuildingResponse> createBuilding({
    // Información básica del edificio
    required String nombreEdificio,
    required String direccion,
    required String ciudad,
    required String codigoPostal,
    required String usoPrincipal,
    required double latitud,
    required double longitud,

    // Información estructural
    required int numeroPisos,
    required double areaTotalPiso,
    required int anioConstruccion,
    required int anioCodigo,
    required bool ampliacion,
    int? anioAmpliacion, // Opcional - Solo si ampliacion es true
    required String ocupacion,
  })

```

Ilustración 1: Captura de código: building_service

3 PASO A PASO DE LA CONEXIÓN

3.1 Backend

1. Instalar dependencias → `npm install`.
2. Levantar servidor → `npm start`.
3. Confirmar que responde en navegador o Postman:
4. <http://192.168.100.19:3000/api/buildings>

3.2 Frontend

1. En el archivo `DatabaseConfig.dart` poner la IP correcta:
2. `static const String baseUrl = 'http://10.0.2.2:3000'; // si es emulador Android`

3. `static const String baseUrl = 'http://192.168.100.19:3000';` // si es celular físico en misma red
4. Ejecutar `flutter pub get`.
5. Correr la app con `flutter run`.
6. La app pedirá los datos al backend y mostrará la información.

3.3 Problemas de conexión

Error mostrado



SIN CONEXIÓN

Error: Sin conexión de red. Verifica que el servidor esté corriendo.

ClientException with SocketException: No route to host (OS

Error: No route to host, errno = 113)

Address: 192.168.100.19

Port: 48062

URL intentada: http://192.168.100.19:3000/auth/login

Ilustracion 2: Captura desde el aplicativo

3.3.1 Análisis del error de conexión

- El error "No route to host" indica que la aplicación no puede establecer conexión con el servidor
- La IP configurada es 192.168.100.19:3000 pero el error muestra puerto 48062
- Posibles causas: servidor apagado, IP incorrecta, problemas de red, o firewall bloqueando la conexión

4 PASOS PARA LA CONFIGURACIÓN DE LA CONEXIÓN

4.1 PASO 1: Verificación de Configuración del Servidor

4.1.1 Verificar configuración de IP en DatabaseConfig

Archivo: lib/core/config/database_config.dart


```

dart

class DatabaseConfig {
  // IP actual configurada
  static const String baseUrl = 'http://192.168.100.19:3000';

  static String getServerUrl() {
    const bool useEmulator = false; // Para dispositivo físico

    if (useEmulator) {
      return 'http://10.0.2.2:3000'; // Para emulador Android
    } else {
      return 'http://192.168.100.19:3000'; // Para dispositivo físico
    }
  }
}

```

Ilustración 3: DatabaseConfig

Instancias a verificar:

1. IP del servidor: Confirmar que **192.168.100.19** es la IP correcta de tu computadora
2. Puerto: Asegurar que el servidor está corriendo en el puerto **3000**
3. Tipo de dispositivo: Si usas emulador, cambiar **useEmulator = true**

4.1.2 Obtener la IP correcta de tu computadora

En Windows:

```

cmd

ipconfig

```

Ilustración 4: PowerShell de Windows

Buscar la línea "Dirección IPv4" en tu adaptador de red activo.

En macOS/Linux:

```
bash
ifconfig
# 0
ip addr show
```

Ilustración 5: Terminal de macOS/Linux

Captura de pantalla esperada:

```
Configuración IP de Windows
Adaptador de Ethernet Wi-Fi:
Dirección IPv4. . . . . : 192.168.100.19
```

Ilustración 6: Terminal de macOS/Linux

4.2 PASO 2: Verificación del Servidor Backend

4.2.1 Confirmar que el servidor está ejecutándose

Comando para verificar:

```
bash

# Si tu servidor es Node.js
npm start
# o
node server.js

# Si es otro tipo de servidor, usar el comando correspondiente
```

Ilustración 7: Comando de verificación del servidor backend

Respuesta esperada:

```
Server running on http://localhost:3000
Database connected successfully
Health endpoint available at /health
```

Ilustración 8: resultado del comando de verificación del servidor backend

4.2.2 Identificar tu arquitectura actual

Según el código, se tiene una configuración híbrida:

1. Supabase en pubspec.yaml:

```
yaml

dependencies:
  supabase_flutter: ^2.10.0
```

Ilustración 9: dependencia de supabase

2. Servidor local en database_config.dart:

dart

```
static const String baseUrl = 'http://192.168.100.19:3000';
```

Ilustración 10: ip del servidor

Esto sugiere que tienes un servidor intermedio (Node.js/Express) que:

- Recibe peticiones de tu app Flutter
- Se comunica con Supabase
- Devuelve respuestas procesadas

4.2.3 Verificar el servidor personalizado

Confirmación necesaria: Basándome en tu configuración, necesitas tener un servidor corriendo en tu computadora local en el puerto 3000.

Posibles ubicaciones del servidor:

1. Carpeta separada del proyecto Flutter (más común)
2. Subcarpeta en tu proyecto Flutter
3. Servidor en Node.js, Python, PHP, etc.

Para encontrar tu servidor:

```
bash

# Buscar archivos de servidor en tu proyecto
find . -name "server.js" -o -name "app.js" -o -name "index.js" -o -name "main.py"

# O buscar carpetas típicas de backend
ls -la | grep -E "(server|backend|api)"
```

Ilustración 11: búsqueda del servidor

Comandos típicos para ejecutar servidor

```
bash

# Node.js
npm start
# o
node server.js
# o
npm run dev

# Python Flask/FastAPI
python app.py
# o
python main.py

# PHP
php -S localhost:3000
```

Ilustración 12: comandos para levantar el servidor

Respuesta esperada al ejecutar el servidor

```
Server running on http://localhost:3000  
Server listening on port 3000  
Ready to accept connections on port 3000  
API server started at http://localhost:3000
```

Ilustración 13: servidor

4.2.4 Probar endpoint de health

- URL a probar: <http://192.168.100.19:3000/health>

Respuestas posibles

Con servidor intermedio:

```
json  
  
{  
  "status": "OK",  
  "message": "Server is running",  
  "supabase": "connected",  
  "timestamp": "2024-01-15T10:30:00.000Z"  
}
```

Ilustración 14: prueba de IP

Error si el servidor no está ejecutándose:

```
Esta página no está disponible  
ERR_CONNECTION_REFUSED
```

Ilustración 15: error del servidor

4.3 PASO 3: Configuración de Red y Firewall

4.3.1 Verificar conectividad de red

Desde la computadora donde corre el servidor:

```
cmd  
  
ping 192.168.100.19
```

Ilustración 16: verificación de conectividad

Respuesta esperada:

```
Haciendo ping a 192.168.100.19 con 32 bytes de datos:  
Respuesta desde 192.168.100.19: bytes=32 tiempo<1m TTL=64
```

Ilustración 17: conectividad exitosa

4.3.2 Verificar que el puerto 3000 esté abierto

Windows (PowerShell):

```
powershell  
  
Test-NetConnection -ComputerName 192.168.100.19 -Port 3000
```

Ilustración 18: Windows PowerShell

Linux/macOS:

```
bash
telnet 192.168.100.19 3000
# 0
nc -zv 192.168.100.19 3000
```

Ilustración 19: Terminal Linux

4.3.3 Configurar Firewall

Paso opcional en caso de ser necesario.

Windows Defender:

1. Abrir "Windows Defender Firewall con seguridad avanzada"
2. Clic en "Reglas de entrada" → "Nueva regla"
3. Seleccionar "Puerto" → "TCP" → "Puertos locales específicos: 3000"
4. Permitir la conexión
5. Aplicar a todos los perfiles

4.4 PASO 4: Prueba de Conexión desde la Aplicación

4.4.1 Usar el widget ConnectionTestAndroid

El archivo `connection_test_android.dart` ya incluye un widget para probar la conexión:


```
dart

// Este widget se muestra en la pantalla de login
const ConnectionTestAndroid()
```

Ilustración 20: prueba de conexión

Estados posibles:

1. ☒ **CONECTADO:**

- Servidor: http://192.168.100.19:3000
- Mensaje: Conexión exitosa con Android
- Respuesta del servidor: Server is running

2. ☒ **SIN CONEXIÓN:**

- Error: Sin conexión de red. Verifica que el servidor esté corriendo.

3. ☒ **ERROR:**

- Error específico mostrado (timeout, network unreachable, etc.)

4.4.2 Interpretar los resultados de la prueba

Conexión exitosa:



CONECTADO

Servidor: `http://192.168.100.19:3000`

Mensaje: Conexión exitosa con Android

Respuesta del servidor: OK

Ilustración 21: conexión con Android Studio

Error de conexión:



SIN CONEXIÓN

Error: Sin conexión de red. Verifica que el servidor esté corriendo.

Error de conexión específico:

ClientException with SocketException: No route to host

Ilustración 22: Error de conexión

4.5 PASO 5: Solución de Problemas Comunes

4.5.1 Error "No route to host"

Soluciones:

1. Verificar IP:

- Obtener IP actual: `ipconfig` (Windows) o `ifconfig` (macOS/Linux)
- Actualizar en `database_config.dart`

2. Verificar que están en la misma red:

- Computadora y dispositivo deben estar en la misma red WiFi
- Verificar que no hay aislamiento de clientes habilitado en el router

3. Reiniciar servicios de red:

```
cmd

# Windows
ipconfig /release
ipconfig /renew
```

Ilustración 23: Reinicio de servicios

4.5.2 Error "Connection timeout"

Soluciones:

1. Aumentar timeout en configuración:

```
dart

static const int connectionTimeout = 30000; // 30 segundos
```

Ilustración 24: timeout de conexión

2. Verificar latencia de red:

```
cmd

ping -t 192.168.100.19
```

Ilustración 25: latencia de red

4.5.3 Error "Connection refused"

Soluciones:

1. Verificar que el servidor está corriendo:

```
bash  
netstat -an | grep 3000
```

Ilustración 26: verificación de servidor

2. Verificar que el servidor escucha en todas las interfaces:

```
app.listen(3000, '0.0.0.0', () => {  
  console.log('Server listening on all interfaces');  
});
```

Ilustración 27: servidor

4.6 PASO 6: Validación Final de la Conexión

4.6.1 Prueba completa del flujo de autenticación

1. Abrir la aplicación
2. Verificar el widget de conexión: Debe mostrar "☒ CONECTADO"
3. Intentar login con credenciales válidas
4. Verificar logs en consola

```
dart

// Logs esperados en Flutter
I/flutter (12345): Intentando conectar a: http://192.168.100.19:3000
I/flutter (12345): Respuesta del servidor: 200
I/flutter (12345): POST Request:
I/flutter (12345):   URL: http://192.168.100.19:3000/auth/login
I/flutter (12345):   Auth required: false
I/flutter (12345): HTTP Response (POST /auth/login):
I/flutter (12345):   Status: 200
I/flutter (12345): Login exitoso en intento 1
```

Ilustración 28: validación de conexión

4.7 Casos de prueba

4.7.1 Prueba 1: Conexión exitosa

- Servidor corriendo ☐
- IP correcta ☐
- Firewall configurado ☐
 - Resultado esperado: Login exitoso

4.7.2 Prueba 2: Servidor apagado

- Detener servidor
- Intentar login
 - Resultado esperado: Error de conexión con mensaje claro

4.7.3 Prueba 3: IP incorrecta

- Cambiar IP en configuración a una IP inexistente

- Intentar login
 - Resultado esperado: Error "No route to host"

4.8 PASO 7: Monitoreo y Logs

4.8.1 Habilitar logs detallados

El `database_service.dart` ya incluye logs detallados:

```
dart

print('GET Request:');
print('  URL: $_baseUrl$endpoint');
print('  Auth required: $requiresAuth');
print('  Has token: ${_authToken != null}');
```

Ilustración 29: monitoreo

4.8.2 Verificar logs del servidor

Asegurar que el servidor backend registre las peticiones:

Checklist de Verificación

- IP del servidor es correcta en `database_config.dart`
- Servidor backend está ejecutándose
- Puerto 3000 está abierto en firewall
- Dispositivo y servidor están en la misma red
- Endpoint `/health` responde correctamente

- Widget ConnectionTestAndroid muestra "☑ CONECTADO"
- Login funciona correctamente
- Logs muestran peticiones exitosas

Información de Debug

Para obtener información de debug de la configuración actual:

```
final debugInfo = DatabaseService.getDebugInfo();  
print('Debug info: $debugInfo');
```

Ilustración 30: debug

Salida esperada:

```
json  
{  
  "baseUrl": "http://192.168.100.19:3000",  
  "hasToken": false,  
  "tokenPreview": null,  
  "connectionTimeout": 30000,  
  "timestamp": "2024-01-15T10:30:00.000Z"  
}
```

Ilustración 31: salida de debug

4.8.3 Conexión exitosa

Con todos estos pasos que no necesariamente debes de hacer todo sino ir probando cada uno y así solucionarás este error común que se puede plantear:



Ilustración 32: aplicativo con conexión exitosa